COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

Quantum Virtual Machine (QVM)

A Hardware-Agnostic Quantum Circuit Execution Platform

A Software Requirements Specification (SRS) Submitted in Partial Fulfillment
of the Requirements
for the Degree of

Bachelor of Science in Computer Science

Submitted By

Muhammad Abdul Qayyoum (CIIT/SP23-BCS-033/VHR)
Ahmad Faraz (CIIT/SP23-BCS-007/VHR)

Supervisor

Assistant Prof. Mr. Farhad Mubashir

Session: Spring 2023-2027

Contents

List of Figures

List of Tables

Chapter 1
Requirement Analysis

1.1 Introduction

This chapter presents a comprehensive analysis of the requirements for the Quantum Virtual Machine (QVM) system. The requirements were gathered through a combination of literature review, analysis of existing quantum computing frameworks, and iterative prototyping. The chapter defines the functional and non-functional requirements that guided the system's design and implementation, ensuring that the QVM achieves its goal of providing a hardware-agnostic quantum execution runtime.

1.2 Requirement Elicitation Techniques

The requirements for the QVM were elicited using multiple complementary techniques to ensure comprehensive coverage of system needs:

Literature Review: Extensive analysis of academic papers and technical documentation from major quantum computing frameworks (Qiskit, Cirq, ProjectQ) to identify industry standards and best practices for quantum compilation and simulation.

Comparative Analysis: Systematic comparison of existing quantum SDKs to identify gaps in portability, educational accessibility, and hardware abstraction that the QVM aims to address.

Prototyping and Iteration: Incremental development approach where initial prototypes revealed technical constraints and user needs, leading to refined requirements for classical memory integration, control flow support, and transpilation strategies.

Domain Expert Consultation: Regular discussions with the FYP supervisor and review of quantum computing research to validate technical requirements and ensure alignment with quantum computing principles.

1.3 User Classes and Characteristics

1.4 System Overview

The Quantum Virtual Machine is a comprehensive quantum circuit execution platform that implements a complete compilation and simulation pipeline. The system accepts quantum programs in multiple formats (OpenQASM 2.0, OpenQASM 3.0, JSON), transforms them through a hardware-agnostic intermediate representation, applies topology-aware transpilation, and executes them using either exact statevector simulation or efficient tensor network methods. The system provides three user interfaces (CLI, Web API, Web GUI) to accommodate different usage scenarios, from automated scripting to interactive exploration. The architecture emphasizes modularity, allowing each component (parser, transpiler, simulator) to be independently tested and upgraded.

Table 1.1: User Classes and Characteristics

User Class	**Description**	**Characteristics**
Quantum Algorithm Developers	Users who write and test quantum algorithms	Technical background in quantum computing; need hardware abstraction; require visualization and debugging tools
Students and Educators	Academic users learning quantum computing concepts	Beginner to intermediate technical skills; need clear documentation; require educational examples and visual feedback
Researchers	Users conducting quantum computing research	Advanced technical skills; need flexibility and extensibility; require accurate simulation and performance metrics
System Integrators	Users integrating QVM with other tools	Software engineering background; need API access; require export capabilities (OpenQASM)

1.5 Use Case Model

This section defines the primary interactions between the actors and the Quantum Virtual Machine system, capturing the functional requirements from a user-centric perspective.

1.5.1 Use Case Diagram

The use case diagram in Figure 1.1 provides a high-level visual overview of the system's functionality and the actors involved.

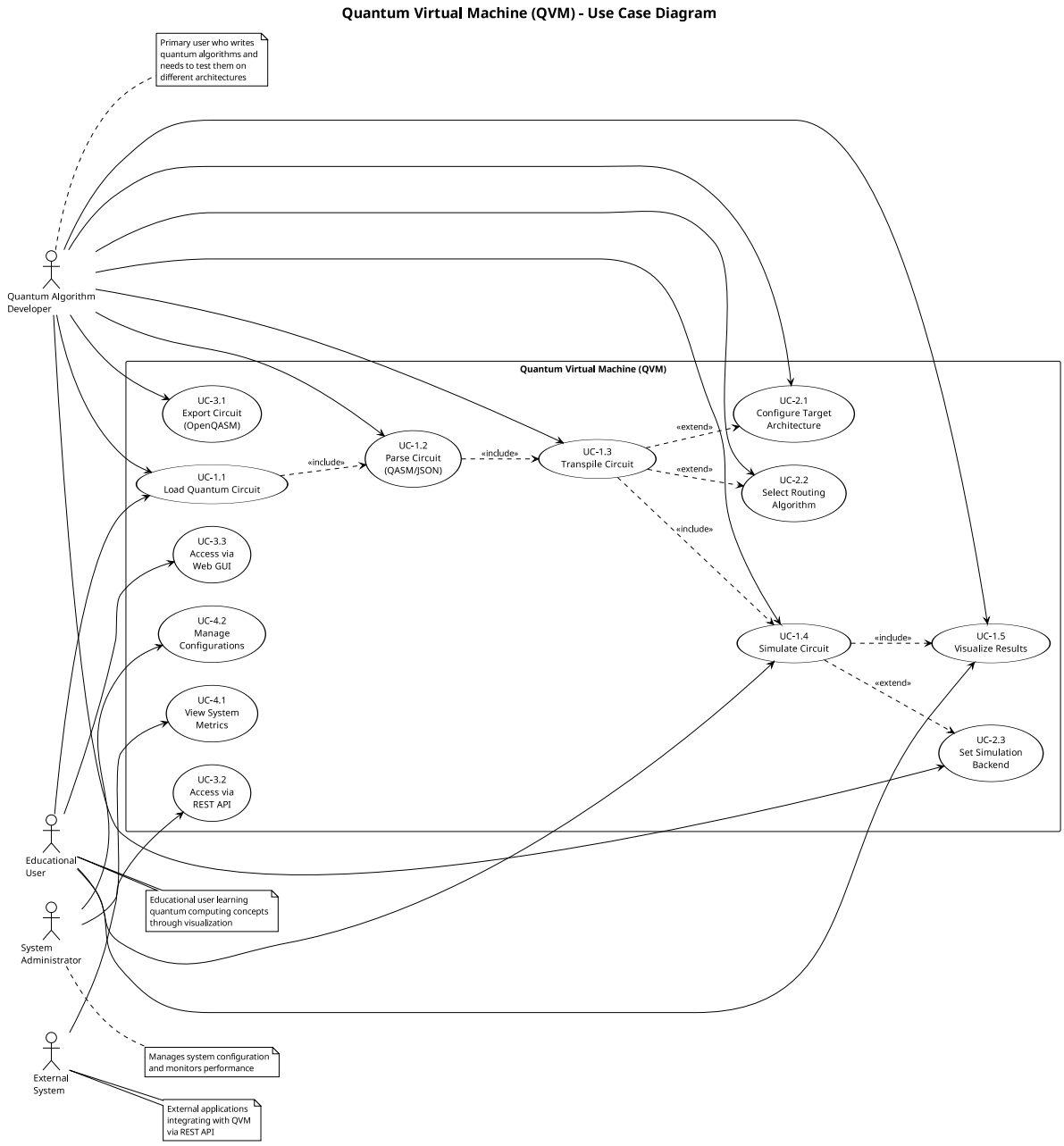

Figure 1.1: Use Case Diagram for Quantum Virtual Machine

1.5.2 Use Case Descriptions

The following descriptions provide detailed step-by-step flows for each use case identified in the diagram, including preconditions, main flows, and alternative paths.

UC-1.1: Parse Quantum Circuit

- **Actor:** Quantum Algorithm Developer
- **Description:** User provides a quantum circuit in OpenQASM or JSON format, and the system parses it into an internal intermediate representation.

- **Preconditions:** Valid circuit file exists
- **Postconditions:** QuantumCircuit IR object is created
- **Main Flow:**
  1. User provides circuit file path
  2. System detects file format (QASM 2.0, QASM 3.0, or JSON)
  3. System invokes appropriate parser
  4. Parser validates syntax and semantics
  5. System creates QuantumCircuit IR object
  6. System returns success confirmation
- **Alternative Flows:**
  - 4a. Syntax error detected: System reports error with line number and description
  - 4b. Semantic error detected: System reports invalid gate or qubit reference

UC-1.2: Transpile Circuit for Target Architecture

- **Actor:** Quantum Algorithm Developer
- **Description:** User requests transpilation of a logical circuit to match a specific hardware topology (e.g., linear chain, grid).
- **Preconditions:** Valid QuantumCircuit IR exists; target architecture is defined
- **Postconditions:** Physical circuit with SWAP gates inserted is generated
- **Main Flow:**
  1. User specifies target architecture and routing strategy
  2. System analyzes circuit connectivity requirements
  3. System applies routing algorithm (Greedy or SABRE)
  4. System inserts SWAP gates to satisfy topology constraints
  5. System optionally restores logical-to-physical mapping
  6. System returns transpiled circuit
- **Alternative Flows:**
  - 2a. Circuit requires more qubits than architecture provides: System reports error
  - 3a. No valid routing path exists: System reports disconnected topology error

UC-1.3: Execute Circuit with Statevector Simulation

- **Actor:** Quantum Algorithm Developer

- **Description:** User executes a quantum circuit using exact statevector simulation.
- **Preconditions:** Valid QuantumCircuit exists; circuit has ≤ 12 qubits
- **Postconditions:** Final statevector and measurement results are returned
- **Main Flow:**
  1. User initiates simulation
  2. System initializes statevector to $|0\rangle^{\otimes n}$
  3. System iterates through circuit operations
  4. For each gate, system applies corresponding unitary matrix
  5. For measurements, system performs probabilistic collapse
  6. System returns final statevector and classical memory
- **Alternative Flows:**
  - 1a. Circuit exceeds 12 qubits: System suggests MPS simulator
  - 3a. Infinite loop detected: System terminates after max operations limit

UC-1.4: Execute Circuit with MPS Simulation

- **Actor:** Researcher
- **Description:** User executes a low-entanglement circuit using Matrix Product State simulation for efficiency.
- **Preconditions:** Valid QuantumCircuit exists; circuit has low entanglement
- **Postconditions:** Approximate statevector and measurement results are returned
- **Main Flow:**
  1. User initiates MPS simulation with bond dimension parameter
  2. System initializes MPS representation
  3. System applies gates using tensor contractions
  4. System performs SVD truncation to maintain bond dimension
  5. System returns final state and truncation error metrics

UC-1.5: Visualize Circuit

- **Actor:** Student
- **Description:** User requests a visual representation of the quantum circuit.
- **Preconditions:** Valid QuantumCircuit exists
- **Postconditions:** Circuit diagram is displayed or saved

- **Main Flow:**
  1. User requests visualization
  2. System generates matplotlib-based circuit diagram
  3. System displays gates on qubit wires with proper spacing
  4. System shows measurement operations and classical bits
  5. System renders diagram to screen or file

UC-1.6: Visualize Measurement Results

- **Actor:** Quantum Algorithm Developer
- **Description:** User requests a histogram of measurement outcome probabilities.
- **Preconditions:** Circuit has been executed; statevector is available
- **Postconditions:** Probability histogram is displayed
- **Main Flow:**
  1. User requests probability visualization
  2. System computes $|\psi_i|^2$ for all basis states
  3. System creates bar chart with basis states on x-axis
  4. System displays probabilities on y-axis
  5. System renders histogram to screen or file

UC-1.7: Export Circuit to OpenQASM

- **Actor:** System Integrator
- **Description:** User exports the internal circuit representation to OpenQASM 2.0 format for use with external tools.
- **Preconditions:** Valid QuantumCircuit exists
- **Postconditions:** OpenQASM string is generated
- **Main Flow:**
  1. User requests OpenQASM export
  2. System generates QASM header with version and includes
  3. System declares qubits and classical registers
  4. System converts each operation to QASM syntax
  5. System returns QASM string

UC-1.8: Execute via Command Line Interface

- **Actor:** Quantum Algorithm Developer
- **Description:** User runs a quantum circuit from the command line with various options.
- **Preconditions:** QVM is installed; circuit file exists
- **Postconditions:** Circuit is executed and results are displayed
- **Main Flow:**
  1. User invokes CLI with circuit file and options
  2. System parses command-line arguments
  3. System loads and parses circuit file
  4. System applies optional transpilation
  5. System executes circuit with specified simulator
  6. System displays results and optional visualizations

UC-1.9: Execute via Web API

- **Actor:** System Integrator
- **Description:** User submits a circuit via HTTP POST request and receives JSON results.
- **Preconditions:** QVM server is running
- **Postconditions:** JSON response with results is returned
- **Main Flow:**
  1. User sends POST request with circuit JSON
  2. Server validates request format
  3. Server parses circuit
  4. Server executes circuit
  5. Server formats results as JSON
  6. Server returns HTTP 200 with results
- **Alternative Flows:**
  - 2a. Invalid JSON: Server returns HTTP 400 with error message
  - 4a. Execution error: Server returns HTTP 500 with error details

UC-1.10: Interact via Web GUI

- **Actor:** Student
- **Description:** User composes and executes circuits through an interactive web dashboard.
- **Preconditions:** QVM server is running; user has web browser

- **Postconditions:** Circuit is executed and results are displayed in browser
- **Main Flow:**
  1. User accesses web interface
  2. User composes circuit using visual editor or text input
  3. User selects execution options (simulator, architecture)
  4. User clicks execute button
  5. System displays circuit diagram
  6. System displays measurement results histogram
  7. System shows execution statistics

1.6 Functional Requirements

The functional requirements define the specific behaviors and capabilities that the QVM system must provide. These requirements are organized by the six major modules of the system architecture: Parsing and Input Processing, Intermediate Representation, Transpilation and Optimization, Simulation Engines, Visualization and Output, and User Interfaces. Each module's requirements specify the operations, validations, and transformations that the system must support to achieve its goal of hardware-agnostic quantum circuit execution.

1.6.1 Module 1: Parsing and Input Processing

The Parsing Module is responsible for converting quantum circuit descriptions from various input formats (OpenQASM 2.0, OpenQASM 3.0, JSON) into the system's internal intermediate representation. This module validates syntax, checks semantic correctness, and ensures that all gate operations and qubit references are well-formed before execution.

1.6.2 Module 2: Intermediate Representation

The Intermediate Representation (IR) Module provides a unified data structure for representing quantum circuits internally. This module supports quantum operations, classical registers, conditional execution, control flow primitives (labels and jumps), and classical bit operations, enabling the system to handle both pure quantum algorithms and hybrid quantum-classical programs.

1.6.3 Module 3: Transpilation and Optimization

The Transpilation Module performs hardware-aware circuit compilation, mapping logical quantum circuits to physical hardware topologies with connectivity constraints. This module implements routing algorithms (Greedy BFS and SABRE) that insert SWAP gates to satisfy topology requirements while minimizing circuit depth and maintaining logical equivalence.

Table 1.2: Functional Requirements - Parsing Module

Req. ID	Description
FR-1.1	The system shall parse OpenQASM 2.0 files with support for standard gates (h, x, y, z, cx, measure)
FR-1.2	The system shall parse OpenQASM 3.0 files with support for control flow (if, for, while) and classical operations
FR-1.3	The system shall parse JSON-formatted gate lists with gate names, qubits, and parameters
FR-1.4	The system shall validate qubit indices to ensure they are within declared range
FR-1.5	The system shall validate gate parameters (e.g., rotation angles) for correct data types
FR-1.6	The system shall report syntax errors with line numbers and descriptive messages
FR-1.7	The system shall support classical register declarations (bit, bit[n])
FR-1.8	The system shall support qubit register declarations (qubit, qubit[n])

1.6.4 Module 4: Simulation Engines

The Simulation Module executes quantum circuits using statevector evolution or tensor network methods. This module supports exact simulation for circuits up to 12 qubits using full statevector representation, and approximate simulation for larger low-entanglement circuits using Matrix Product State (MPS) decomposition with configurable bond dimensions.

1.6.5 Module 5: Visualization and Output

The Visualization Module generates graphical representations of quantum circuits and measurement results. This module creates circuit diagrams showing gate operations on qubit wires, probability histograms for measurement outcomes, and exports circuits to OpenQASM 2.0 format for interoperability with external quantum computing platforms.

1.6.6 Module 6: User Interfaces

The User Interface Module provides three access methods for interacting with the QVM system: a command-line interface for scripting and automation, a RESTful web API for programmatic access, and a browser-based graphical interface for interactive circuit composition and execution. These interfaces accommodate different user workflows from batch processing to educational exploration.

Table 1.3: Functional Requirements - IR Module

Req. ID	Description
FR-2.1	The system shall represent quantum circuits as a QuantumCircuit object with num_qubits and operations list
FR-2.2	The system shall store operations as dictionaries containing gate name, qubits, parameters, and conditions
FR-2.3	The system shall support conditional operations with register, index, and value specifications
FR-2.4	The system shall support measurement operations with target classical bit specification
FR-2.5	The system shall support label and jump operations for control flow
FR-2.6	The system shall support classical operations (assignment, AND, OR, XOR, NOT)
FR-2.7	The system shall support delay operations with duration specifications
FR-2.8	The system shall provide string representation of circuits for debugging

1.7 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the QVM system must satisfy. These requirements ensure that the system not only provides the correct functionality but also meets standards for performance, reliability, usability, portability, maintainability, and security. The following subsections detail the specific non-functional requirements across these six quality dimensions.

1.7.1 Performance Requirements

Performance requirements specify the execution speed and resource efficiency that the system must achieve. These requirements ensure that the QVM can simulate quantum circuits within acceptable time bounds on standard computing hardware, making it practical for educational use and algorithm development.

1.7.2 Reliability Requirements

Reliability requirements define the system's ability to handle errors gracefully and maintain correct operation under various conditions. These requirements ensure that the QVM validates inputs, detects potential issues like infinite loops, and provides clear diagnostic information when problems occur.

1.7.3 Usability Requirements

Usability requirements specify the ease of use and learnability of the system for different user classes. These requirements ensure that the QVM provides clear documentation, intuitive inter-

Table 1.4: Functional Requirements - Transpiler Module

Req. ID	Description
FR-3.1	The system shall support Greedy BFS routing strategy for qubit mapping
FR-3.2	The system shall support SABRE routing strategy with lookahead heuristics
FR-3.3	The system shall insert SWAP gates to satisfy topology connectivity constraints
FR-3.4	The system shall support linear chain topology (qubits connected in a line)
FR-3.5	The system shall support grid topology (qubits connected in 2D lattice)
FR-3.6	The system shall optionally restore logical-to-physical qubit mapping after transpilation
FR-3.7	The system shall validate that logical circuit fits within target architecture qubit count
FR-3.8	The system shall cache distance calculations for performance optimization
FR-3.9	The system shall decompose complex gates (Toffoli) into native gate sets
FR-3.10	The system shall preserve single-qubit gates without modification during transpilation

faces, and helpful examples that enable users to quickly understand and effectively use the system for quantum algorithm development and education.

1.7.4 Portability Requirements

Portability requirements define the system's ability to run on different operating systems and integrate with external quantum computing platforms. These requirements ensure that the QVM can be deployed across Windows, Linux, and macOS environments, and can exchange circuit definitions with industry-standard tools through OpenQASM format support.

1.7.5 Maintainability Requirements

Maintainability requirements specify the code quality standards and development practices that facilitate future modifications and extensions. These requirements ensure that the QVM codebase is well-documented, follows consistent coding conventions, and uses modular design principles that enable developers to understand, modify, and extend the system efficiently.

1.7.6 Security Requirements

Security requirements define the system's protection mechanisms against malicious inputs and resource exhaustion attacks. These requirements ensure that the QVM validates all user inputs,

Table 1.5: Functional Requirements - Simulator Module

Req. ID	Description
FR-4.1	The system shall simulate circuits using exact statevector evolution for up to 12 qubits
FR-4.2	The system shall support single-qubit gates (H, X, Y, Z, Rx, Ry, Rz, S, T, SX)
FR-4.3	The system shall support two-qubit gates (CX, SWAP)
FR-4.4	The system shall support three-qubit gates (CCX/Toffoli)
FR-4.5	The system shall perform measurements with probabilistic state collapse
FR-4.6	The system shall maintain classical memory registers synchronized with quantum state
FR-4.7	The system shall support conditional gate execution based on classical bit values
FR-4.8	The system shall support control flow (labels, jumps, loops) with infinite loop detection
FR-4.9	The system shall support MPS simulation for low-entanglement circuits with configurable bond dimension
FR-4.10	The system shall report truncation error for MPS simulations
FR-4.11	The system shall use NumPy vectorization for efficient matrix operations
FR-4.12	The system shall support seeded random number generation for reproducible measurements

prevents directory traversal vulnerabilities, and implements resource limits to protect against denial-of-service attempts while maintaining usability for legitimate users.

1.8 External Interface Requirements

External interface requirements specify how the QVM system interacts with users, hardware, software components, and external systems. These requirements define the boundaries of the system and the protocols for communication across those boundaries. The following subsections detail the user interfaces, hardware interfaces, software dependencies, and communication protocols that the system must support.

1.8.1 User Interfaces

The system provides three user interfaces:

Command-Line Interface (CLI): A terminal-based interface accepting file paths and command-line flags. Supports batch processing and scripting. Displays text-based output with optional visualization windows.

Web API: A RESTful HTTP API built with FastAPI. Accepts JSON POST requests at /execute endpoint. Returns JSON responses with statevector, probabilities, and execution metadata. Supports CORS for cross-origin requests.

Table 1.6: Functional Requirements - Visualization Module

Req. ID	Description
FR-5.1	The system shall generate circuit diagrams using matplotlib with gates on qubit wires
FR-5.2	The system shall display measurement operations with classical bit targets
FR-5.3	The system shall generate probability histograms for measurement outcomes
FR-5.4	The system shall export circuits to OpenQASM 2.0 format
FR-5.5	The system shall save visualizations to PNG files
FR-5.6	The system shall display execution statistics (circuit depth, gate count)

Table 1.7: Functional Requirements - Interface Module

Req. ID	Description
FR-6.1	The system shall provide a command-line interface accepting file paths and options
FR-6.2	The system shall support CLI flags for transpilation (–transpile, –routing, –architecture)
FR-6.3	The system shall support CLI flags for simulation (–simulator, –shots, –seed)
FR-6.4	The system shall provide a REST API with POST /execute endpoint
FR-6.5	The system shall accept JSON circuit definitions via API
FR-6.6	The system shall return JSON responses with statevector, probabilities, and metadata
FR-6.7	The system shall provide a web GUI with circuit editor
FR-6.8	The system shall display real-time execution results in web GUI
FR-6.9	The system shall provide example circuits in web GUI

Web GUI: A browser-based dashboard with circuit editor, execution controls, and result visualization. Built with HTML/CSS/JavaScript. Communicates with backend via REST API. Displays circuit diagrams and probability histograms inline.

1.8.2 Hardware Interfaces

The system runs on standard computing hardware with the following minimum specifications:

- CPU: Dual-core processor (2.0 GHz or higher)
- RAM: 4 GB minimum (8 GB recommended for 12-qubit circuits)
- Storage: 100 MB for installation
- Display: 1024x768 resolution for GUI

Table 1.8: Non-Functional Requirements - Performance

Req. ID	Description
NFR-1.1	The system shall simulate 10-qubit circuits in under 5 seconds on standard hardware
NFR-1.2	The system shall parse OpenQASM 3.0 files in under 1 second for circuits with up to 1000 operations
NFR-1.3	The system shall transpile circuits with under 100 gates in under 2 seconds using SABRE routing
NFR-1.4	The system shall support MPS simulation of 20-qubit low-entanglement circuits
NFR-1.5	The system shall cache distance calculations to avoid redundant BFS searches

Table 1.9: Non-Functional Requirements - Reliability

Req. ID	Description
NFR-2.1	The system shall detect and prevent infinite loops in control flow with configurable operation limits
NFR-2.2	The system shall validate all input circuits before execution
NFR-2.3	The system shall provide descriptive error messages for all failure modes
NFR-2.4	The system shall maintain numerical stability for statevector normalization
NFR-2.5	The system shall achieve 95% test coverage with automated unit tests

1.8.3 Software Interfaces

The system interfaces with the following software components:

Python Runtime: Requires Python 3.10 or higher for execution.

NumPy: Used for linear algebra operations (matrix multiplication, tensor products). Version 1.21+ required.

Matplotlib: Used for visualization (circuit diagrams, histograms). Version 3.5+ required.

Lark: Used for OpenQASM 3.0 parsing with LALR parser. Version 1.1+ required.

FastAPI: Used for web API server. Version 0.95+ required.

Uvicorn: ASGI server for running FastAPI application.

1.8.4 Communication Interfaces

The system uses the following communication protocols:

HTTP/HTTPS: Web API communicates via HTTP POST requests with JSON payloads. Supports standard HTTP status codes (200, 400, 500).

File I/O: Reads circuit files from local filesystem. Supports .qasm, .json file extensions. Writes visualization outputs to .png files.

Standard I/O: CLI uses stdin for input and stdout/stderr for output. Supports piping and

Table 1.10: Non-Functional Requirements - Usability

Req. ID	Description
NFR-3.1	The system shall provide comprehensive documentation with usage examples
NFR-3.2	The system shall provide clear CLI help messages with –help flag
NFR-3.3	The system shall provide example circuits for common algorithms (Bell, GHZ, Grover)
NFR-3.4	The system shall use intuitive gate naming conventions consistent with Qiskit
NFR-3.5	The web GUI shall be accessible without installation via web browser

Table 1.11: Non-Functional Requirements - Portability

Req. ID	Description
NFR-4.1	The system shall run on Windows, Linux, and macOS operating systems
NFR-4.2	The system shall require only Python 3.10+ and standard scientific libraries
NFR-4.3	The system shall export circuits to OpenQASM 2.0 for compatibility with IBM Quantum
NFR-4.4	The system shall support multiple input formats (QASM 2.0, QASM 3.0, JSON)

redirection.

1.9 Requirement Traceability Matrix

1.10 Summary

This chapter has presented a comprehensive analysis of the Quantum Virtual Machine requirements, covering functional requirements across six major modules (Parsing, IR, Transpilation, Simulation, Visualization, Interfaces) and non-functional requirements across six quality attributes (Performance, Reliability, Usability, Portability, Maintainability, Security). The requirements were elicited through literature review, comparative analysis, and iterative prototyping. Ten detailed use cases were documented, covering the primary user interactions with the system. The requirement traceability matrix demonstrates clear alignment between business objectives and functional requirements. These requirements guided the system design and implementation, ensuring that the QVM achieves its goal of providing a hardware-agnostic, educational, and lightweight quantum execution runtime.

Table 1.12: Non-Functional Requirements - Maintainability

Req. ID	**Description**
NFR-5.1	The system shall follow modular architecture with clear separation of concerns
NFR-5.2	The system shall include docstrings for all public functions and classes
NFR-5.3	The system shall use type hints for function signatures
NFR-5.4	The system shall maintain version control with Git
NFR-5.5	The system shall follow PEP 8 Python style guidelines

Table 1.13: Non-Functional Requirements - Security

Req. ID	**Description**
NFR-6.1	The system shall validate all file paths to prevent directory traversal attacks
NFR-6.2	The system shall sanitize user input in web API to prevent injection attacks
NFR-6.3	The system shall limit maximum circuit size to prevent resource exhaustion
NFR-6.4	The system shall implement operation count limits to prevent denial-of-service

Table 1.14: Requirement Traceability Matrix

Business Objective	**Functional Requirements**	**Use Cases**
BO-1: Hardware Agnosticism	FR-3.1, FR-3.2, FR-3.3, FR-3.4, FR-3.5	UC-1.2
BO-2: Educational Value	FR-5.1, FR-5.2, FR-5.3, FR-6.7, FR-6.9	UC-1.5, UC-1.6, UC-1.10
BO-3: Lightweight Runtime	FR-4.1, FR-4.11, NFR-1.1, NFR-4.2	UC-1.3
BO-4: Interoperability	FR-1.1, FR-1.2, FR-5.4, NFR-4.3	UC-1.1, UC-1.7
BO-5: Flexibility	FR-2.3, FR-2.5, FR-2.6, FR-4.7, FR-4.8	UC-1.3, UC-1.4